n8n-nodes-dag-orchestrator

Blueprint-Style DAG Workflow Orchestrator

n8n Community Node Package

Development Specification Document v1.0

April 2026

1. Project Overview

n8n-nodes-dag-orchestrator is an n8n community node that brings Unreal Engine Blueprint-style visual workflow orchestration to n8n. It solves the most commonly reported pain points in the n8n community: parallel execution, dependency management, synchronization, nested loops, and error handling — all within a single, self-contained node.

> **Core Problem Being Solved**
>
> n8n users currently need multiple sub-workflows, manual index tracking, and complex workarounds to achieve parallel execution with dependencies. This node eliminates all of that by providing a single, powerful orchestration engine inside one node.

1.1 Package Information

Property	Value
Package Name	n8n-nodes-dag-orchestrator
Node Display Name	DAG Orchestrator
Version	0.1.0
Node.js Requirement	v22 or higher
n8n API Version	1
License	MIT
Keywords	n8n-community-node-package, dag, parallel, orchestrator, blueprint

1.2 Inspiration

The design is inspired by Unreal Engine's Blueprint visual scripting system, where users connect execution nodes in a graph, define dependencies visually, and group related logic into collapsible subgraphs. This same philosophy is applied to n8n workflow orchestration.

2. Core Features

2.1 Parallel Branch Execution

The node can execute multiple branches simultaneously. Branches that have no dependencies on each other run in parallel, dramatically improving performance over n8n's default sequential execution.

- Binary data branches and JSON data branches can run at the same time
- Each branch operates independently in its own execution context
- Results are collected and merged at a configurable join point

2.2 Directed Acyclic Graph (DAG) Dependency Management

Users can define dependencies between branches. A branch only starts executing when all of its declared dependencies have successfully completed. This mirrors how professional data pipeline orchestrators like Apache Airflow and Dagster work.

- Branch A can depend on Branch B's output before starting
- Dependency chain is resolved automatically at runtime
- Circular dependency detection prevents infinite loops
- Dependency data is passed as input to downstream branches automatically

2.3 Try / Catch / Finally Blocks

Every branch has three execution phases inspired by standard programming error handling:

Block	Purpose
try	Main operation — the primary logic to execute for this branch
catch	Error handler — runs only if the try block fails. Can retry, log, or use fallback data
finally	Cleanup — always runs regardless of success or failure. Used for temp file cleanup, releasing resources, etc.

2.4 Retry Logic with Configurable Delays

Each branch can be configured with automatic retry behaviour, solving one of the most frequently requested features in the n8n community.

- Maximum retry count configurable per branch
- Delay between retries in milliseconds
- Exponential backoff option for API rate limit handling
- Different retry strategies for 4xx vs 5xx errors

2.5 Timeout Handling

Each branch has its own timeout setting. If a branch exceeds its timeout:

- The branch is marked as timed out
- Downstream branches that depend on it receive a timeout signal
- A configurable skip-on-timeout option allows dependent branches to proceed with fallback data

2.6 Nested / Collapsible Branch Groups

Branches can be grouped into named subgraphs. This keeps complex workflows readable and manageable without needing separate sub-workflows.

- Groups are collapsed by default in the node UI
- Expanding a group shows the internal branch structure
- Groups can contain their own loops, conditionals, and error handlers
- Eliminates the need for sub-workflows for most common use cases

2.7 Loop Management with Index Tracking

The node provides first-class loop support with automatic index tracking, removing the need for manual run index hacks.

- Automatic currentIndex and totalItems tracking inside loops
- Support for nested loops without the known n8n nested loop bug
- Loop over arrays, paginated API responses, or file lists
- Break and continue conditions configurable per loop

2.8 Conditional Branch Execution

Branches can have entry conditions. A branch only executes if its condition evaluates to true. This is similar to Airflow's ShortCircuitOperator.

- Conditions can reference outputs from previous branches
- Support for skip-on-false or fail-on-false modes
- Multiple conditions can be combined with AND or OR logic

2.9 Data Type Support

Data Type	Handling
JSON	Passed between branches as structured objects. Supports dot-notation access.
Binary	Passed by reference using internal memory buffers. Avoids the sub-workflow binary loss bug.
Mixed	Both types can coexist in the same branch execution context.
Transformed	Branches can transform data before passing it downstream.

2.10 Connection Strategy (Internal Memory)

To avoid connection crashes and n8n's parallel execution limitations, all inter-branch data passing happens through an internal in-memory state object. Only the final merged output connects outward to the next node in the workflow.

- No physical n8n connections between branches — everything is internal
- State is scoped to the current execution — no cross-execution contamination
- Binary data stored in memory buffers, not serialised and re-parsed

- Single output connection from the node carries all branch results

3. Real-World Example — Image Processing

This is the concrete use case that drove the design of this node. You receive an image file and need to process it in multiple dependent ways.

Scenario

Input: Binary image file. Goal: Store the image to drive, extract metadata, generate a thumbnail — with proper dependency ordering and error handling throughout.

3.1 Branch Structure

Branch	Description
Branch 1 — Image Storage	FIRST to run. No dependencies. Takes binary image, saves to drive, outputs file path.
Branch 2 — Metadata Extraction	Depends on Branch 1. Takes binary image, extracts dimensions and file size. Receives Branch 1 path.
Branch 3 — Thumbnail Generation	Depends on BOTH Branch 1 and Branch 2. Creates thumbnail using metadata and stored path.
Join Point	Waits for all three branches. Merges results into one output object for the next node.

3.2 Error Handling Per Branch

Branch 1 — Image Storage

- try: Save binary to Google Drive, return file path
- catch: Log error, store placeholder path like /error/no-file
- finally: Delete any temp files created during upload

Branch 2 — Metadata Extraction

- try: Extract image dimensions, file size, MIME type
- catch: Return default metadata object with null values
- finally: Release image buffer from memory
- timeout: 30 seconds — skip thumbnail if exceeded

Branch 3 — Thumbnail Generation

- try: Generate 200x200 thumbnail, save to drive

- catch: Log failure, mark thumbnail as unavailable
- finally: Clean up resize buffers
- condition: Only runs if Branch 1 AND Branch 2 both succeeded

3.3 Configuration Schema (Simplified)

```
{
  "branches": [
    {
      "id": "branch_1",
      "name": "Image Storage",
      "dependencies": [],
      "dataType": "binary",
      "try": { "operation": "storeFile", "destination": "googleDrive" },
      "catch": { "operation": "logError", "fallback": { "path": "/error/no-
file" } },
      "finally": { "operation": "cleanup" },
      "timeout": 60000,
      "retry": { "maxAttempts": 3, "delayMs": 5000 }
    },
    {
      "id": "branch_2",
      "name": "Metadata Extraction",
      "dependencies": ["branch_1"],
      "dataType": "binary",
      "try": { "operation": "extractMetadata" },
      "catch": { "operation": "defaultMetadata" },
      "finally": { "operation": "releaseBuffer" },
      "timeout": 30000,
      "skipOnTimeout": true
    },
    {
      "id": "branch_3",
      "name": "Thumbnail Generation",
      "dependencies": ["branch_1", "branch_2"],
      "condition": { "allSucceeded": ["branch_1", "branch_2"] },
      "try": { "operation": "generateThumbnail", "size": "200x200" },
      "catch": { "operation": "markUnavailable" },
      "finally": { "operation": "cleanup" }
    }
  ],
  "joinMode": "waitForAll",
  "outputFormat": "merged"
}
```

4. Architecture

4.1 Execution Engine

The DAG Orchestrator uses an internal execution engine that resolves the dependency graph at runtime and schedules branches accordingly. The engine follows these steps on each execution:

1. Parse branch configuration and build the dependency graph
2. Topologically sort branches to determine execution order

3. Identify which branches have no dependencies and can start immediately
4. Execute ready branches in parallel using Promise.all
5. As each branch completes, check which dependent branches are now unblocked
6. Continue until all branches have completed or failed
7. Collect all outputs and merge into the final result

4.2 State Management

Each execution maintains an isolated state object containing:

- branchResults: Map of branch ID to output data
- branchStatus: Map of branch ID to status (pending, running, success, failed, timeout, skipped)
- binaryBuffers: In-memory buffer store for binary data
- loopState: Current index, total items, and accumulated results for active loops
- executionLog: Ordered log of all branch events for debugging

4.3 Data Flow

Stage	Data Handling
Input	Node receives n8n items. Binary and JSON are separated and stored in state.
Branch Execution	Each branch receives a context object containing its dependency outputs and the original input.
Inter-Branch Passing	Upstream branch outputs are injected into downstream branch context automatically.
Output	All branch results are merged into a single n8n item array and passed to the next node.

4.4 Subgraph Nesting

Branch groups are implemented as nested DAG instances. When the engine encounters a group, it instantiates a child execution engine, runs the subgraph, and returns the group result as a single branch output to the parent graph.

- Infinite nesting depth is theoretically supported
- Each group has its own isolated state
- Group timeout wraps all internal branch timeouts
- Group-level try/catch wraps the entire subgraph execution

5. Node Configuration

5.1 Top-Level Settings

Setting	Description
Execution Mode	parallel (default) or sequential
Join Mode	waitForAll, waitForFirst, or waitForAny
Output Format	merged (single object), array (per-branch results), or passthrough
Timeout (global)	Global timeout in ms. Overrides individual branch timeouts if exceeded.
Error Strategy	stopOnFirst, continueOnError, or collectErrors

5.2 Per-Branch Settings

Setting	Description
id	Unique identifier for this branch. Used for dependency references.
name	Human-readable label shown in the node UI.
dependencies	Array of branch IDs that must complete before this branch starts.
condition	Optional condition object. Branch skips if condition is false.
dataType	json, binary, or mixed.
try	The main operation configuration.
catch	Error handler configuration.
finally	Cleanup configuration.
timeout	Branch-level timeout in milliseconds.
skipOnTimeout	If true, dependent branches receive fallback data instead of failing.
retry.maxAttempts	Number of retry attempts on failure.
retry.delayMs	Delay between retry attempts in milliseconds.
retry.backoff	none, linear, or exponential.

6. Loop Handling

The node provides a dedicated loop configuration that eliminates the most common n8n loop issues: nested loop bugs, manual index tracking, and loop breaks not working.

6.1 Loop Configuration

Property	Description
source	The array or paginated source to iterate over.

batchSize	Number of items to process per iteration.
indexVar	Variable name for the current index in branch expressions.
totalVar	Variable name for total item count in branch expressions.
breakCondition	Expression evaluated after each iteration. Breaks loop if true.
continueCondition	Expression evaluated before each iteration. Skips iteration if false.
maxIterations	Hard limit to prevent infinite loops.

6.2 Nested Loops

Groups containing loops can be nested inside outer loops without triggering the known n8n nested loop bug, because each group maintains its own loop state rather than relying on n8n's global run index.

Example: Folder Loop

Loop over a folder of image files. For each file, run a subgroup containing the three image processing branches (storage, metadata, thumbnail). No sub-workflow needed. All handled internally.

7. Error Handling Strategy

7.1 Branch-Level Error Flow

Each branch follows this error resolution order:

8. try block executes
9. If try fails, retry up to maxAttempts with configured delay
10. If all retries fail, catch block executes
11. If catch block also fails, branch is marked as failed
12. finally block always executes after try or catch
13. Parent branches receive the failed status and respond per their condition settings

7.2 Global Error Strategies

Strategy	Behaviour
stopOnFirst	The entire node execution stops on the first branch failure. Other running branches are cancelled.
continueOnError	Failed branches are recorded but execution continues. Join receives partial results.
collectErrors	All branches run to completion. Final output includes a dedicated errors array with all failures.

8. Project File Structure

```
n8n-nodes-dag-orchestrator/
├── nodes/
│   └── DagOrchestrator/
│       ├── DagOrchestrator.node.ts     # Main node definition
│       ├── engine/
│       │   ├── DagEngine.ts            # Core DAG execution engine
│       │   ├── BranchExecutor.ts       # Individual branch runner
│       │   ├── StateManager.ts         # Execution state store
│       │   ├── DependencyResolver.ts   # Topological sort + validation
│       │   └── LoopController.ts       # Loop index and iteration logic
│       ├── types/
│       │   ├── Branch.types.ts         # Branch config interfaces
│       │   ├── Dag.types.ts            # DAG config interfaces
│       │   └── State.types.ts          # State object interfaces
│       └── icon.svg
├── credentials/                        # Add credentials here if needed
├── package.json
├── tsconfig.json
├── tsconfig.build.json
└── README.md
```

9. Development Phases

Phase 1 — Core Engine (Week 1–2)

- Set up package boilerplate with TypeScript and n8n types
- Implement DependencyResolver with topological sort
- Implement basic parallel execution with Promise.all
- Implement StateManager for branch results
- Basic try/catch/finally per branch
- Unit tests for dependency resolution

Phase 2 — Data Handling (Week 3–4)

- Binary buffer management (in-memory, no serialisation)
- JSON data passing between branches
- Mixed data type support
- Input parsing from n8n items
- Output merging to n8n items

Phase 3 — Error Handling and Retry (Week 5–6)

- Retry logic with configurable delays
- Exponential backoff

- Timeout handling per branch
- Skip-on-timeout with fallback data
- Global error strategies (stopOnFirst, continueOnError, collectErrors)

Phase 4 — Loop Support (Week 7–8)

- Loop configuration and index tracking
- Break and continue conditions
- Nested loop support with isolated state
- Maximum iteration guard

Phase 5 — Subgraphs and Groups (Week 9–10)

- Nested branch group implementation
- Child engine instantiation and result propagation
- Group-level timeout and error wrapping
- UI representation of collapsed groups

Phase 6 — Node UI and Polish (Week 11–12)

- Conditional branch execution based on upstream results
- n8n node UI parameter definitions
- Execution log output for debugging
- README and example workflows
- Lint, build, and publish pipeline with GitHub Actions provenance

10. Publishing Requirements

Based on current n8n community node requirements, the following must be in place before publishing:

- Package name must start with n8n-nodes-
- n8n-community-node-package must be in package.json keywords
- Nodes and credentials registered in package.json under the n8n attribute
- npm run lint must pass with zero warnings
- README must include installation instructions and example workflow

Important Deadline

From May 1st 2026, verified nodes must be published via GitHub Actions with a provenance statement. A GitHub Actions publish workflow (.github/workflows/publish.yml) must be configured before that date to remain eligible for verification.

11. Known Limitations and Constraints

- Verified community nodes cannot use runtime npm dependencies — all logic must be self-contained
- Binary data is held in memory for the duration of execution — very large files may cause memory pressure
- The node UI in n8n does not support true visual graph editing — branch configuration is JSON-based in the parameters panel
- n8n cloud compatibility requires the node to pass the n8n Creator Portal verification process

12. References

- n8n Community Node Starter: github.com/n8n-io/n8n-nodes-starter
- n8n Building Nodes Documentation: docs.n8n.io/integrations/community-nodes/build-community-nodes
- n8n Creator Portal: creators.n8n.io
- Apache Airflow DAG Concepts: airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html
- Dagster Conditional Branching: docs.dagster.io
- Unreal Engine Blueprint System: docs.unrealengine.com/blueprint-visual-scripting

End of Document